



## Desarrollo de Aplicaciones Informáticas

# Examen - JavaScript, Objetos y DOM

---

**Docente:** Ing. Pablo Morandi

**Ayudante:** Matías Marchesi

**Curso:** 5to año - Informática

**Duración:** 2 horas cátedra (80 min)

**Fecha:** 06/07/2026

---

## EXAMEN PARCIAL — TEMA 1B

---

### Leer con atención antes de empezar el examen:

**Material permitido:** Apuntes de clase, documentación oficial. El uso de la IA implica que **el examen sea anulado automáticamente** sin excepción y con nota 1(unos).

**Formato de entrega:** Subir carpeta completa del proyecto en un archivo .RAR con el formato

`"5x_DAI_APELLIDO"` a Google Classroom.

#### IMPORTANTE:

1. Se entrega un archivo `index.html` base con Bootstrap ya incluido.
  2. Crear y solo trabajar en un archivo `script.js` que ya está importado en `index.html`.
- 

## Calculadora de Envíos

Este examen tiene **3 bloques independientes**. Se puede empezar por cualquiera.

**RESTRICCIÓN:** Está **prohibido** usar los métodos `find()`, `filter()`, `sort()` y `findIndex()`. Implementar toda búsqueda o recorrido con `for`, `while` o `forEach`.

---

## Bloque 1 — JavaScript puro

### Ejercicio 1.1 — Funciones de costo

Crear las siguientes funciones de costo:

- `costoPorPeso(kg)` : devuelve `kg * 1500`
  - `costoPorDistancia(km)` : devuelve `km * 85.5`
  - `costoInternacional(pesos, cotizacion)` : devuelve `pesos / cotizacion`
- 

### Ejercicio 1.2 — Función redondear

Crear la función `redondear(numero, decimales)` que:

1. Calcule el factor: `const factor = Math.pow(10, decimales)`
2. Devuelva `Math.round(numero * factor) / factor`

Probar en la consola: `redondear(costoPorPeso(2.5), 2)` debe dar `3750`, y `redondear(costoInternacional(10000, 1350), 3)` debe dar `7.407`.

---

## Bloque 2 — Clases

### Ejercicio 2.1 — Clase Envio

Crear la clase `Envio` con los siguientes atributos en el constructor:

- `id` (number)
  - `tipo` (string): `"peso"`, `"distancia"` o `"internacional"`
  - `valorOriginal` (number)
  - `costo` (number)
  - `fecha` (string): usar `new Date().toLocaleDateString()`
- 

### Ejercicio 2.2 — Clase HistorialEnvios

Crear la clase `HistorialEnvios` con los siguientes atributos en el constructor:

- `envios` (array): inicializar vacío `[]`

Y los siguientes métodos:

- `agregar(envio)` : agrega una instancia de `Envio` al array `envios`
  - `obtenerPorTipo(tipo)` : recorre `envios` con un `for`, acumula en un array los envíos cuyo `tipo` coincida y lo devuelve
- 

### Ejercicio 2.3 — Datos de prueba

Crear una instancia global del historial: `const historial = new HistorialEnvios()`.

Agregar **3 envíos** (recuerde que son instancias de la clase **Envio**) de prueba usando

`historial.agregar()`:

- id:1, tipo: "peso", valorOriginal: 2.5, costo: 3750
- id:2, tipo: "distancia", valorOriginal: 12, costo: 1026
- id:3, tipo: "internacional", valorOriginal: 10000, costo: 7.41

Crear también un contador `nextId = 4`.

## Bloque 3 — DOM

### Ejercicio 3.1 — Formulario de cotización

Crear la función `cotizarEnvio()` que:

1. Obtenga el tipo de envío desde un `<select id="select-tipo">` con opciones: `"peso"`, `"distancia"`, `"internacional"`
2. Obtenga el valor desde un `<input id="input-valor">` (convertir a `Number`)
3. Valide que el valor no sea `NaN` ni vacío. Si no se cumple alguna, mostrar un **alert**.
4. Calcule el costo según el tipo:
  - `"peso"`: use `costoPorPeso(valor)`
  - `"distancia"`: use `costoPorDistancia(valor)`
  - `"internacional"`: use `costoInternacional(valor, 1350)` (cotización fija)
5. Redondee el resultado con `redondear(resultado, 2)`
6. Cree un nuevo `Envio` con `nextId` e incremente `nextId`
7. Llame a `historial.agregar(envio)`
8. Muestre el resultado en el elemento con `id="resultado"`
9. Llame a `mostrarEnvios()` para actualizar la lista

### Ejercicio 3.2 — Mostrar historial

Crear la función `mostrarEnvios()` que:

1. Seleccione el elemento con `id="lista-historial"`
2. Recorra `historial.envios` con un `for`
3. Por cada envío, muestre tipo, valor original, costo y fecha con este formato: `${tipo} - ${valorOriginal} → $$${costo} (${fecha})` e insertelo dentro de una etiqueta `<p>`
4. Asigne el HTML al `innerHTML` del elemento

Llamar a `mostrarEnvios()` con `window.onload`.

### Ejercicio 3.3 — Limpiar historial

Crear la función `limpiarEnvios()` que:

1. Vacíe el array `historial.envios` asignándole `[]`
2. Llame a `mostrarEnvios()` para actualizar el DOM